



Autonomous Data Analyst Agent

Complete Technical Specification, Source Code & Cloud Deployment Manual

Project Status: **LIVE IN PRODUCTION**

Live Public URL: <https://autonomous-data-analyst-507.streamlit.app>

Deployment Target: Streamlit Community Cloud

GitHub Repository: [singamalaomsai/autonomous-data-analyst-agent](https://github.com/singamalaomsai/autonomous-data-analyst-agent)

1. System Architecture & Capabilities Overview

The **Autonomous Data Analyst Agent** is an intelligent conversational business intelligence application engineered to automate the end-to-end data exploration, numerical computation, and graphical visualization lifecycle. It combines state-of-the-art Large Language Models (LLMs) via the Groq Inference Engine with a secure Python execution runtime.

Layer / Component	Technology	Core Responsibility
Frontend UI & State	Streamlit 1.32+	Interactive multi-session chat UI, sidebar data loader, dynamic chart and table rendering.
LLM Inference	Groq API (<code>llama-3.3-70b-versatile</code>)	Context-aware query decomposition, Pandas query synthesis, and natural language insights.
Execution Sandbox	Python <code>exec()</code> + <code>contextlib</code> + <code>io.StringIO</code>	Isolated runtime executing synthesized Pandas / NumPy logic with captured stdout and figure streams.
Visualization Engine	Matplotlib & Seaborn	Automated chart generation (bar, line, scatter, histograms) seamlessly rendered to Streamlit.
Cloud Infrastructure	Streamlit Community Cloud	Containerized Linux runtime with automated GitHub CI/CD tracking branch <code>main</code> .

2. Complete Project Directory Structure

The production repository was selectively curated to contain only essential production scripts, datasets, and environment definitions:

Project Directory Layout

```
autonomous-data-analyst-agent/  
├── .streamlit/  
│   └── secrets.toml          # Local API secrets (strictly gitignored)  
├── .gitignore                # Excludes virtual environments and sensitive credentials  
├── analyst_tools.py          # Sandbox execution runtime and dataset inspection engine  
├── app.py                    # Main Streamlit web application with Groq integration  
├── company_metrics.csv       # Sample dataset: enterprise departmental KPIs & revenue  
├── store_sales_data.csv      # Sample dataset: retail transactions and product performance  
├── requirements.txt          # Python dependency specification for cloud containers  
├── README.md                 # Public project documentation & live demo portal  
└── PROJECT_WORKFLOW.md      # Comprehensive deployment records and operational logs
```

3. Complete Source Code Implementation

3.1 Dependency Manifest (`requirements.txt`)

```
requirements.txt
```

```
streamlit>=1.32.0
pandas>=2.0.0
numpy>=1.24.0
matplotlib>=3.7.0
seaborn>=0.12.0
groq>=0.5.0
```

3.2 Version Control Exclusions (`.gitignore`)

```
.gitignore
```

```
# Python Virtual Environments
.venv/
venv/
ENV/
env/

# Bytecode & Caches
__pycache__/
*.py[.cod]
*$py.class
.pytest_cache/

# Streamlit Local Secrets & Configuration
.streamlit/secrets.toml

# Environment Variables & IDE configs
.env
.vscode/
.idea/
.DS_Store
Thumbs.db
```

3.3 Local Secrets Specification (`.streamlit/secrets.toml`)

```
.streamlit/secrets.toml
```

```
# Groq Cloud API Authentication Key
GROQ_API_KEY = "gsk_your_actual_groq_api_key_here"
```

3.4 Tool Execution Engine & Sandbox (`analyst_tools.py`)

This module provides the core analytical capabilities: inspecting arbitrary DataFrames to extract structural metadata and executing generated Python code blocks within a controlled namespace.

```
analyst_tools.py
```

```
import io
import sys
import contextlib
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

def get_dataset_overview(df: pd.DataFrame) -> dict:
    buffer = io.StringIO()
    df.info(buf=buffer)
    info_str = buffer.getvalue()

    return {
        "columns": list(df.columns),
        "shape": df.shape,
        "dtypes": {k: str(v) for k, v in df.dtypes.items()},
```

```

        "missing_values": df.isnull().sum().to_dict(),
        "info_summary": info_str,
        "head": df.head(5).to_dict(orient="records")
    }

def execute_python_code(code_string: str, df: pd.DataFrame) -> dict:
    stdout_capture = io.StringIO()
    fig_created = False
    plt.close('all')

    local_vars = {
        "df": df.copy(),
        "pd": pd,
        "np": np,
        "plt": plt,
        "sns": sns
    }

    try:
        with contextlib.redirect_stdout(stdout_capture):
            exec(code_string, {}, local_vars)

        output_text = stdout_capture.getvalue()

        # Check if matplotlib created any active figures
        if plt.get_fignums():
            fig_created = True

        return {
            "success": True,
            "output": output_text.strip() if output_text.strip() else "Code executed successfully with no
print output.",
            "fig": plt.gcf() if fig_created else None
        }
    except Exception as e:
        return {
            "success": False,
            "output": f"Runtime Execution Error: {str(e)}",
            "fig": None
        }

```

3.5 Streamlit Web Application (app.py)

The user-facing frontend handles dataset upload, dynamic prompts, session state history, Groq API communication, code block parsing, and live interactive rendering.

app.py

```

import os
import streamlit as st
import pandas as pd
from groq import Groq
from analyst_tools import get_dataset_overview, execute_python_code

st.set_page_config(
    page_title="Autonomous Data Analyst Agent",
    page_icon="📊",
    layout="wide"
)

# Authenticate via Streamlit Secrets (Cloud) or Environment Variables (Local)
api_key = st.secrets.get("GROQ_API_KEY", os.getenv("GROQ_API_KEY"))

if not api_key:
    st.error("Groq API Key not detected. Please configure GROQ_API_KEY in Secrets.")
    st.stop()

```

```

client = Groq(api_key=api_key)

st.title("📊 Autonomous Data Analyst Agent")
st.caption("Powered by Groq LLM, Pandas, & Tool-Calling Runtime Execution")

# Sidebar - Dataset Management
st.sidebar.header("📁 Data Source")
uploaded_file = st.sidebar.file_uploader("Upload CSV Dataset", type=["csv"])

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.sidebar.success(f"Loaded: {uploaded_file.name} ({df.shape[0]} rows, {df.shape[1]} cols)")
else:
    default_path = "company_metrics.csv"
    if os.path.exists(default_path):
        df = pd.read_csv(default_path)
        st.sidebar.info(f"Loaded default: `{default_path}`")
    else:
        st.sidebar.warning("Please upload a CSV dataset to begin.")
        df = None

# Initialize Chat Session History
if "messages" not in st.session_state:
    st.session_state.messages = []

# Render Existing Conversation
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])
        if "figure" in msg and msg["figure"] is not None:
            st.pyplot(msg["figure"])

# Handle User Prompt
if prompt := st.chat_input("Ask a question about the dataset..."):
    if df is None:
        st.warning("Please upload or select a CSV dataset first.")
        st.stop()

    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    with st.chat_message("assistant"):
        overview = get_dataset_overview(df)

        system_prompt = f"""
You are an expert Autonomous Data Analyst Agent.
Dataset Overview:
- Columns: {overview['columns']}
- Data Types: {overview['dtypes']}
- Shape: {overview['shape']}
- Sample Preview: {overview['head']}

Instructions:
1. Formulate a rigorous analytical strategy to answer the user request.
2. Write clean, executable Python code operating strictly on the `df` DataFrame.
3. Use `print()` to display numeric/text findings or `plt`/`sns` to generate visualizations.
4. Wrap all executable code in a single ```python ... ``` block.
"""

        response = client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": prompt}
            ],
            temperature=0.2

```

```

)

agent_reply = response.choices[0].message.content
st.markdown(agent_reply)

fig = None
if "```python" in agent_reply:
    code_block = agent_reply.split("```python")[1].split("```")[0].strip()
    with st.expander("🔗 View Agent Executed Code"):
        st.code(code_block, language="python")

    result = execute_python_code(code_block, df)
    if result["success"]:
        if result["output"]:
            st.success("Execution Output:")
            st.text(result["output"])
        if result["fig"]:
            fig = result["fig"]
            st.pyplot(fig)
    else:
        st.error(result["output"])

st.session_state.messages.append({
    "role": "assistant",
    "content": agent_reply,
    "figure": fig
})

```

4. Step-by-Step Terminal & Git Operations

1 Local Virtual Environment Activation

Set PowerShell execution policies and activate the isolated virtual environment:

```

(Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt

```

2 Staging Production Artifacts & Identifying Practice Files

Selectively staged production files while leaving demo files untracked:

```

git add app.py requirements.txt .gitignore analyst_tools.py company_metrics.csv store_sales_data.csv
git status

```

3 Committing Production Build & Initializing Branch

```

git commit -m "Deploy Autonomous Data Analyst Agent"
git branch -M main

```

5. Authentication Challenges & Resolution Protocols

Encountered Error: Git 403 Forbidden Access

```
remote: Permission to singamalaomsai/autonomous-data-analyst-agent.git denied to
singamalaomsairedy507-cell.
fatal: unable to access 'https://github.com/...': The requested URL returned error: 403
```

4 Root Cause Analysis & Windows Credential Manager Fix

Windows OS stored cached OAuth tokens under a secondary GitHub account (`singamalaomsairedy507-cell`). Git automatically presented these invalid credentials to GitHub.

- Opened **Control Panel** → **Credential Manager** → **Windows Credentials**.
- Located and removed the cached generic credential for `git:https://github.com` .

5 Explicit Username Remote Binding

Configured Git remote URL to explicitly mandate the `singamalaomsai` identity, triggering browser OAuth re-authentication:

```
git remote set-url origin https://singamalaomsai@github.com/singamalaomsai/autonomous-data-analyst-agent.git
git push -u origin main
```

6 Removing Practice Files from Git Index Cache

To ensure repository cleanliness without deleting local files from disk, indexed demo files were purged via `--cached` :

```
git rm --cached check1.py check2.txt check3.doc check4.html help.txt hi.py hii.txt hiii.html sai.py we.doc wee.docx
git commit -m "Remove practice demo files"
git push origin main
```

6. Streamlit Community Cloud Deployment Lifecycle

7 Connecting Streamlit Community Cloud

- Authenticated to **share.streamlit.io** using the primary GitHub account `singamalaomsai` .
- Selected **Deploy an existing app** and mapped:
 - **Repository:** `singamalaomsai/autonomous-data-analyst-agent`
 - **Branch:** `main`
 - **Main file path:** `app.py`
 - **Subdomain:** `autonomous-data-analyst-507`

8 Production Secrets Injection

Navigated to **Advanced Settings** → **Secrets** in Streamlit Cloud dashboard and injected the encrypted Groq API key:

```
GROQ_API_KEY = "gsk_..."
```

Production Status: Application Active & Live Worldwide

The container compiled dependencies from `requirements.txt` in 82 seconds and launched at:

7. Repository Documentation & Verification Summary

The repository was finalized with clear documentation (`README.md` and `PROJECT_WORKFLOW.md`) enabling cross-device mobile verification and public collaboration:

Verification Vector	Methodology	Observed Outcome
Mobile Browser Access	Opened live URL in Chrome/Safari on smartphone	Responsive UI rendered cleanly with functional chat interface and dataset switching.
Dataset Query Execution	Prompted: <i>"Analyze department revenue and plot bar chart"</i>	Agent generated valid Pandas aggregation, executed script, and displayed chart.
Secret Isolation	Inspected public GitHub repository files	<code>secrets.toml</code> correctly ignored; API credentials completely secured.